

Python 编程与人工智能

引言

Chengwei QIN

助理教授 @AI Thrust

1. 课程信息
2. Python 简介
3. Python 与 IDE 配置
4. GitHub Copilot 配置
5. 简单的 LLM API 调用演示
6. Python 基础



课程信息

课程安排: Sep 1, 2026 – Dec 7, 2026

- 时间: 周一 04:30PM - 05:50PM, 周五 12:00PM - 01:20PM
- 地点: A 讲堂

授课教师: Chengwei QIN

TA:

- Hehai LIN, hlin709@connect.hkust-gz.edu.cn
- Jiayu YANG, jyang729@connect.hkust-gz.edu.cn

其余待定。



Chengwei QIN

LLM 推理

智能体

多模态

高效学习

■ 助理教授@AI Thrust, 从事 NLP 与 LLM 研究

■ 本科毕业于 THU, 博士毕业于 NTU

论文奖项

- ❑ WAIC 青年优秀论文奖入围名单, 2026 (LLM Reasoning)
- ❑ EMNLP Outstanding Paper Award, 2024 (LLM Reasoning)
- ❑ ACL Area Chair Award, 2023 (Multimodality)

其他经历

- ❑ Meta AI、AWS AI

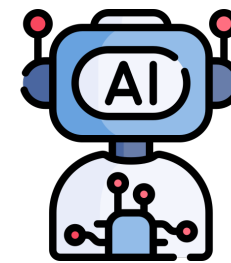
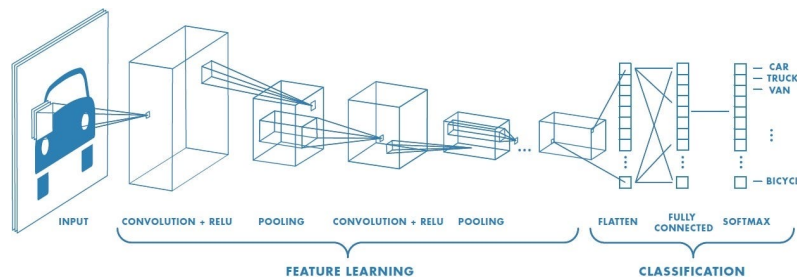
Chengwei QIN, 邮箱: chengweiqin@hkust-gz.edu.cn

答疑时间: 周五 1:30pm - 2:30pm, W3-310, 或预约

课程目标

完成本课程后，学生将能够：

- 夯实高级 Python 编程概念的基础
- 学习使用 Python 实现核心机器学习和深度学习模型
- 通过整合模型、数据流水线、API 与评估工作流，熟练构建基于 Python 的 AI 应用
- 动手实践现代 AI 辅助软件开发工具，并学会在项目开发中有效使用它们



■ 课程内容概览

周次	标题
1 ~ 2	Python 基础
2 ~ 4	面向数据科学与机器学习的 Python 包 (scikit-learn)
4 ~ 6	深度学习 (例如视觉模型以及包括 LLM 在内的语言模型)
7	AI 智能体。
8	Embodied AI。
9	Coding Agent。
10 ~ 12	软件工程与 AI
13	期末项目展示——汇报、反思、总结。

■ 评分方案

课程组成	权重 (%)
出勤	10%
课堂练习	20%
作业 (3 次)	30% (各 10%)
期末小组项目	40%

前两周不考勤，课堂练习也不计分。

出勤 (自 2026 年 9 月 15 日起，选课/退课期结束后)

- 每堂课都会记录出勤。

课堂练习 (自 2026 年 9 月 15 日起，选课/退课期结束后)

- 每堂课都会包含一些简短练习
- 每堂课 1-2 次练习会计入成绩
- 请务必完成并提交这些计分练习

每人有 2 次考勤免扣缺勤，以及 2 次课堂练习免扣缺勤。

这是小组项目。每组应由 2-4 名学生组成。

- 设计并构建一个有趣且有创意的系统
- 撰写报告，说明你的设计与发现
- 课堂展示：分享系统设计、做演示，并突出你的贡献
- 提交代码以便评测与复现

截止日期 -- 第 13 周第一次课前一天的 23:59 (L01 为 Dec. 3)

期末项目选题建议将在接下来几周公布。

累计分数 (满分 100)	可能的成绩
90~100	A-/A/A+
70~90	B-/B/B+/A-
50~70	C-/C/C+/B-
40~50	D/C-
<40	F

注意：

1. 考核主要看预期学习成果的达成情况。
2. 以上对应关系来自以往经验，应视为初步参考。

- 请勿抄袭。学术诚信至关重要。任何抄袭都将导致严重后果。
- 教学方式：约 60% 讲授概念/示例 + 约 40% 练习

- 我们对生成式 AI 工具持开放态度。
- 这是一门基础编程课，重要目标之一是培养你们自己的编程能力
- 理解底层原理有助于更好地与 AI 沟通，并更有效地使用 AI 工具。
- 记住：AI 应辅助思考，而非取代思考

“我无法创造的东西，我就不理解。”

——Richard Feynman



还有问题？



Python 简介

最流行的编程语言之一

<https://www.tiobe.com/tiobe-index/>

TIOBE 编程社区指数是衡量编程语言流行度的指标。

Aug 2026	Aug 2025	Change	Programming Language		Ratings	Change
1	1		 Python		18.53%	-7.61%
2	3	▲	 C		11.10%	+2.07%
3	2	▼	 C++		8.62%	-0.56%
4	4		 Java		8.25%	-0.34%
5	5		 C#		4.09%	-1.43%
6	6		 JavaScript		2.63%	-0.52%
7	7		 Visual Basic		2.18%	-0.15%
8	12	▲	 SQL		1.88%	+0.16%
9	14	▲	 R		1.56%	+0.19%
10	18	▲	 Rust		1.45%	+0.31%

为什么用 Python

对初学者友好、入门级的语言，最初即为教育而设计

- 可读性高
 - 在英语中有意义的关键字
 - 用换行结束命令，用缩进定义作用域
- 快速开发与实验
 - 丰富的库支持：PyPI 上有超过 600,000 个包
 - 高级内置功能（用 C 实现）
 - 脚本语言，带交互模式，便于快速测试与调试
- 跨平台兼容：可在 **macOS、Linux、Windows** 乃至 **Raspberry Pi** 上无缝运行
- 社区庞大活跃：教程、资源与支持丰富

■ 缺点

牺牲效率换取简洁

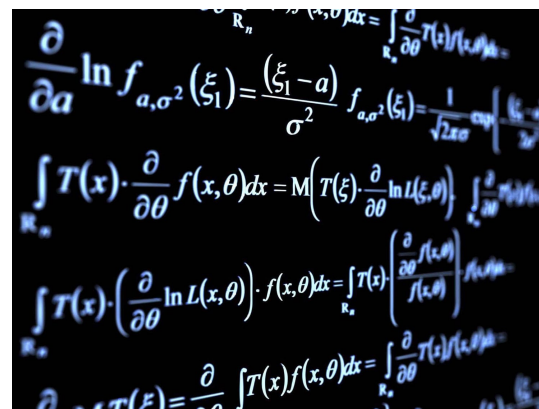
- 执行较慢：基本操作运行效率较低
- 内存占用更高：在内存优化至关重要时并不理想（例如移动应用）

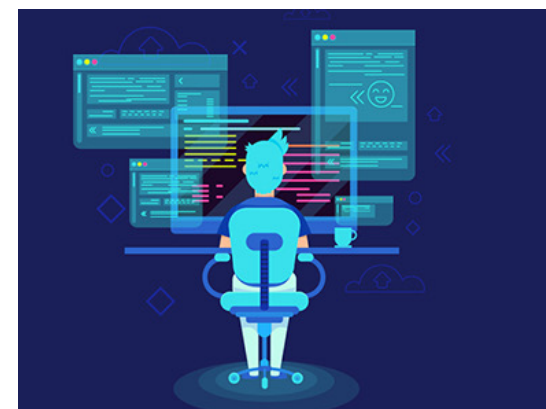
解决方案：将性能关键组件用 **C/C++** 实现，再通过 **Python** 接口对外提供

Python 适用于.....

实际部署中对效率没有要求的任何场景

- 研究
 - 数据处理、数据分析、计算、...
- 自动化脚本
 - OS、数据库、...
- 产品原型开发
- 机器学习/深度学习
- 网络爬虫


$$\begin{aligned}\frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) &= \frac{(\xi_1 - a)}{\sigma^2} f_{a, \sigma^2}(\xi_1) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\xi_1 - a)^2}{2\sigma^2}\right) \\ \int T(x) \cdot \frac{\partial}{\partial \theta} f(x, \theta) dx &= M\left(T(\xi) \cdot \frac{\partial}{\partial \theta} \ln L(\xi, \theta)\right) \\ \int T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) \cdot f(x, \theta) dx &= \int T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln f(x, \theta)\right) f(x, \theta) dx \\ \frac{\partial}{\partial \theta} \ln L(\xi) &= \frac{\partial}{\partial \theta} \int T(x) f(x, \theta) dx = \int T(x) \frac{\partial}{\partial \theta} f(x, \theta) dx\end{aligned}$$





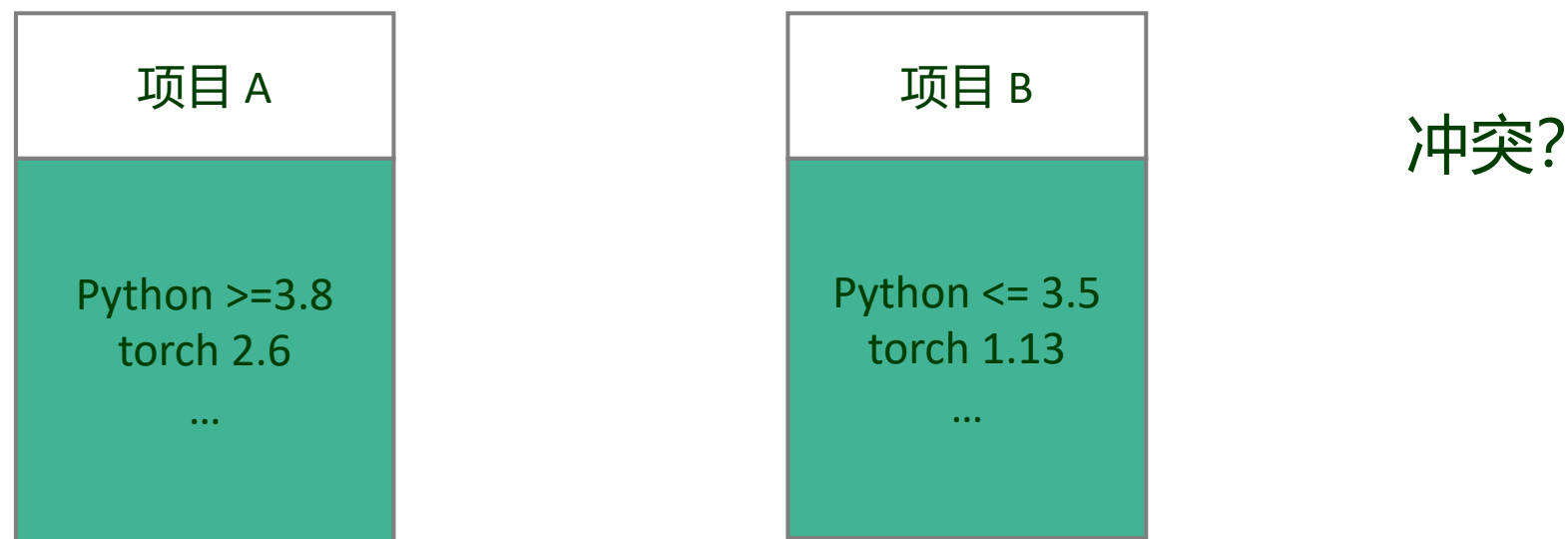
Python 与 IDE 配置

既然大家已有编程基础（甚至已用过 Python），我们将跳过部分基础内容并快速过一遍。如有需要请课后自行阅读；环境配置遇到困难随时问我，我会帮忙。

安装 Anaconda (Anaconda: <https://www.anaconda.com/>) 后, 你将获得:

- **Python** — 用于运行 Python 程序的解释器。
- 许多常用 Python 包——例如 NumPy、pandas 和 Matplotlib。
- **Conda** — 用于创建和管理多个隔离 Python 环境的工具, 有助于避免不同 Python 版本与包之间的冲突。

<https://docs.conda.io/projects/conda/en/latest/user-guide/install/index.html>



Conda 命令

安装 anaconda 后，打开命令提示符窗口：

- Windows：搜索 Anaconda Prompt
- macOS：打开终端
- Linux：打开终端

在 Prompt 窗口中：

- 创建环境
- 激活环境
- 安装软件包
- 卸载软件包
- 删除环境

```
# create an environment
```

```
conda create -n env1 python=3.6
```

```
# switch the environment
```

```
conda activate -n env1
```

```
# install packages
```

```
conda install seaborn
```

```
# remove an environment
```

```
conda remove -n env1 --all
```

示例：对斐波那契数列做一些分析

- 斐波那契数列：1, 1, 2, 3, 5, 8, ...
 - $a[n] = a[n-1] + a[n-2]$
- 分析
 - `print a[1000000000]`

示例：对斐波那契数列做一些分析

- 斐波那契数列：1, 1, 2, 3, 5, 8, ...

- $a[n] = a[n-1] + a[n-2]$

- 分析

- `print a[1000000000]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
```

执行代码需 31 秒

示例：对斐波那契数列做一些分析

- 斐波那契数列：1, 1, 2, 3, 5, 8, ...

- $a[n] = a[n-1] + a[n-2]$

- 分析

- `print a[1000000000]`

- 打印 `a[999999999] + a[1002]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
9  print(lst[999999999] + lst[1002])
```

再花 31 秒执行代码

交互模式可以节省时间

```
Python 3.9.13 (main, Aug 25 2022, 18:29:29)
[Clang 12.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> lst = [0, 1]
>>> for u in range(2, 1000000001):
...     lst.append((lst[u-1] + lst[u-2]) % 10000000000)
...
>>> print(lst[1000000000])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
>>> print(lst[100000000])
7760546875
>>> print(lst[99999])
6278790626
>>> print(lst[99999] + lst[1002])
8451423002
>>> print(lst[9999999] * lst[10000])
7364937187632913750
>>>
```

处理耗时 31 秒
然后再进行分析

下次如何复用这段代码？

交互模式与脚本模式

交互式	文件
即时反馈	没有即时反馈
逐行运行	执行整个文件
不可复用	可复用
难以调试	易于调试

观察：

- 交互模式适合快速试想法，但不适合较大项目。
- 脚本模式结构清晰、可复用，但即时反馈不够灵活

动机：

能否结合两者的优点？

这就引出了 Jupyter Notebook。

混合方式：以代码块编写，结合脚本模式与交互模式

耗时
数据处理

交互式分析

无需重新执行处理即可添加
分析

```
jupyter_example.ipynb > print(lst[9])
+ 代码 + Markdown | ▶ 全部运行 | 清除所有单元格输出 | Outline ...
py37 (Python 3.7.13)
```

```
[1]: lst = []
    for i in range(100000000):
        lst.append(i)
    print(len(lst))
... 100000000 Python
```

```
[3]: print(lst[99])
... 99 Python
```

```
[4]: print(lst[9])
... 9 Python
```

✓ 交互式计算

✓ 可复用脚本

Jupyter Notebook 入门

安装 Jupyter Notebook

```
Shell

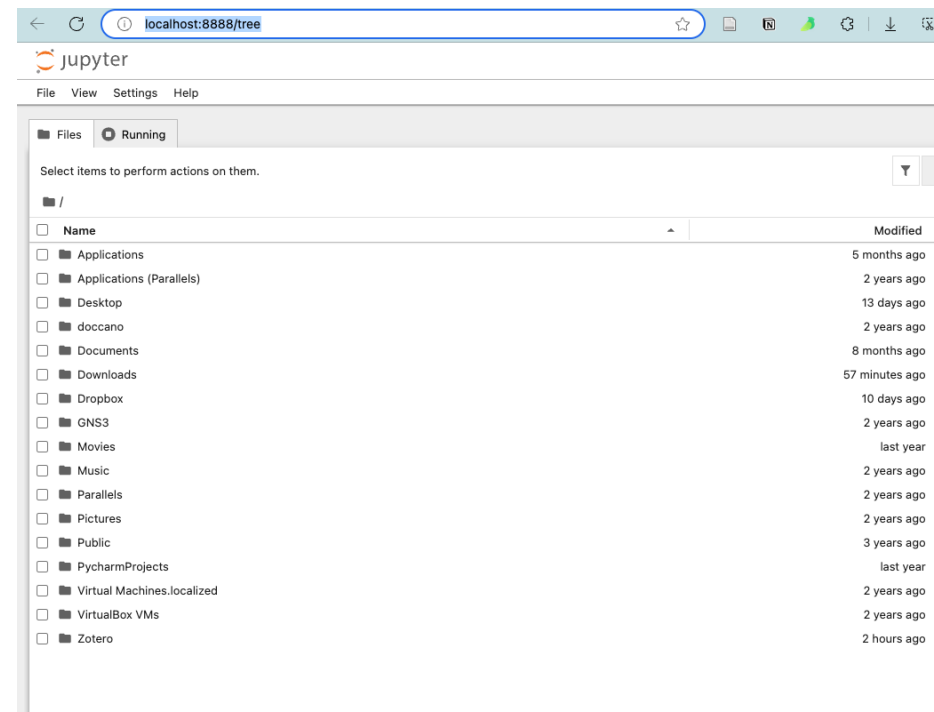
$ pip install jupyter
```

运行 Jupyter Notebook

```
jupyter-notebook
[I 2025-08-26 16:39:25.790 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2025-08-26 16:39:25.791 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2025-08-26 16:39:25.793 ServerApp] jupyterlab | extension was successfully linked.
[I 2025-08-26 16:39:25.794 ServerApp] notebook | extension was successfully linked.
[I 2025-08-26 16:39:25.943 ServerApp] notebook_shim | extension was successfully linked.
[I 2025-08-26 16:39:25.963 ServerApp] notebook_shim | extension was successfully loaded.
[I 2025-08-26 16:39:25.964 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2025-08-26 16:39:25.965 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab extension loaded from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/jupyterlab
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab application directory is /Library/Frameworks/Python.framework/Versions/3.13/share/jupyter/lab
[I 2025-08-26 16:39:25.967 LabApp] Extension Manager is 'pypi'.
[I 2025-08-26 16:39:25.996 ServerApp] jupyterlab | extension was successfully loaded.
[I 2025-08-26 16:39:25.998 ServerApp] notebook | extension was successfully loaded.
[I 2025-08-26 16:39:25.999 ServerApp] Serving notebooks from local directory: /Users/chenhuangxun
[I 2025-08-26 16:39:25.999 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2025-08-26 16:39:25.999 ServerApp] http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2025-08-26 16:39:26.003 ServerApp]

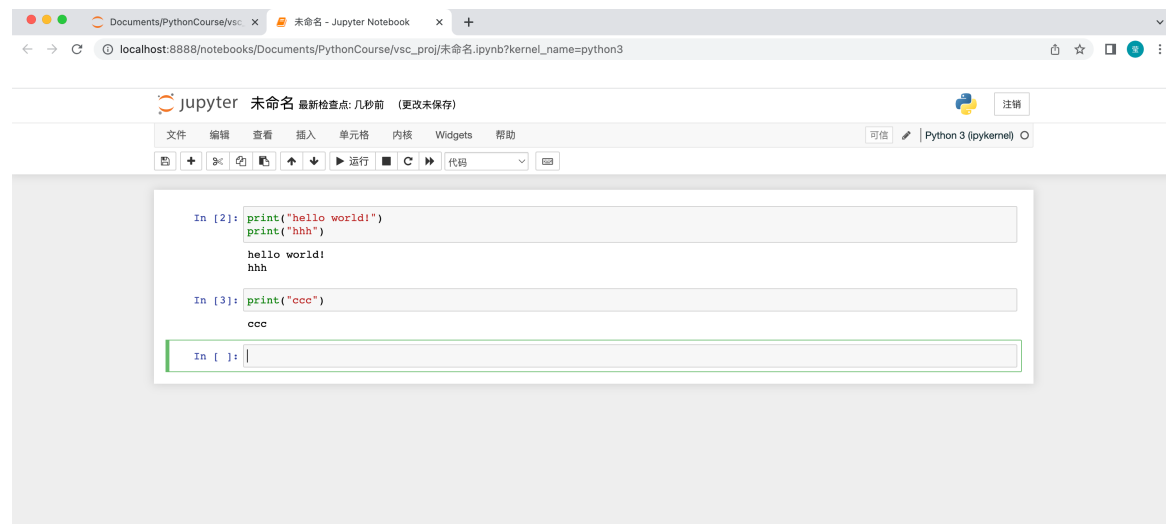
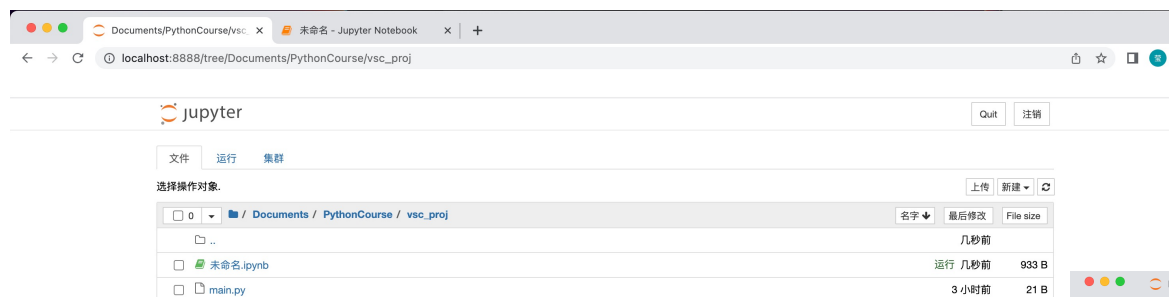
To access the server, open this file in a browser:
file:///Users/chenhuangxun/Library/Jupyter/runtime/jpserver-76649-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:26.012 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
```

在浏览器中打开 Jupyter



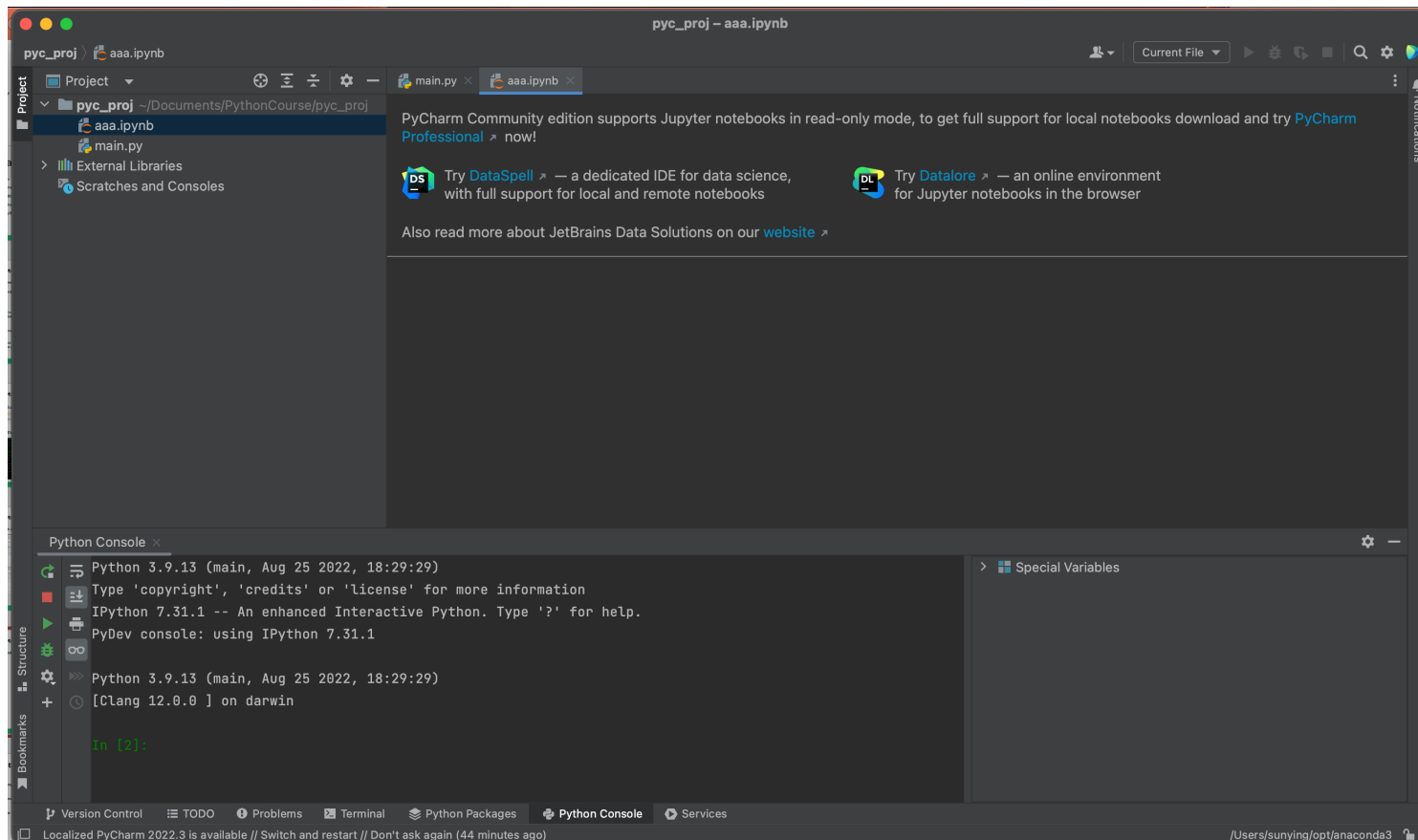
Jupyter Notebook 入门

创建一个 ipynb 文件并尝试
在各个单元格中编写并运行代码
在浏览器中即时查看结果



Jupyter Notebook (PyCharm)

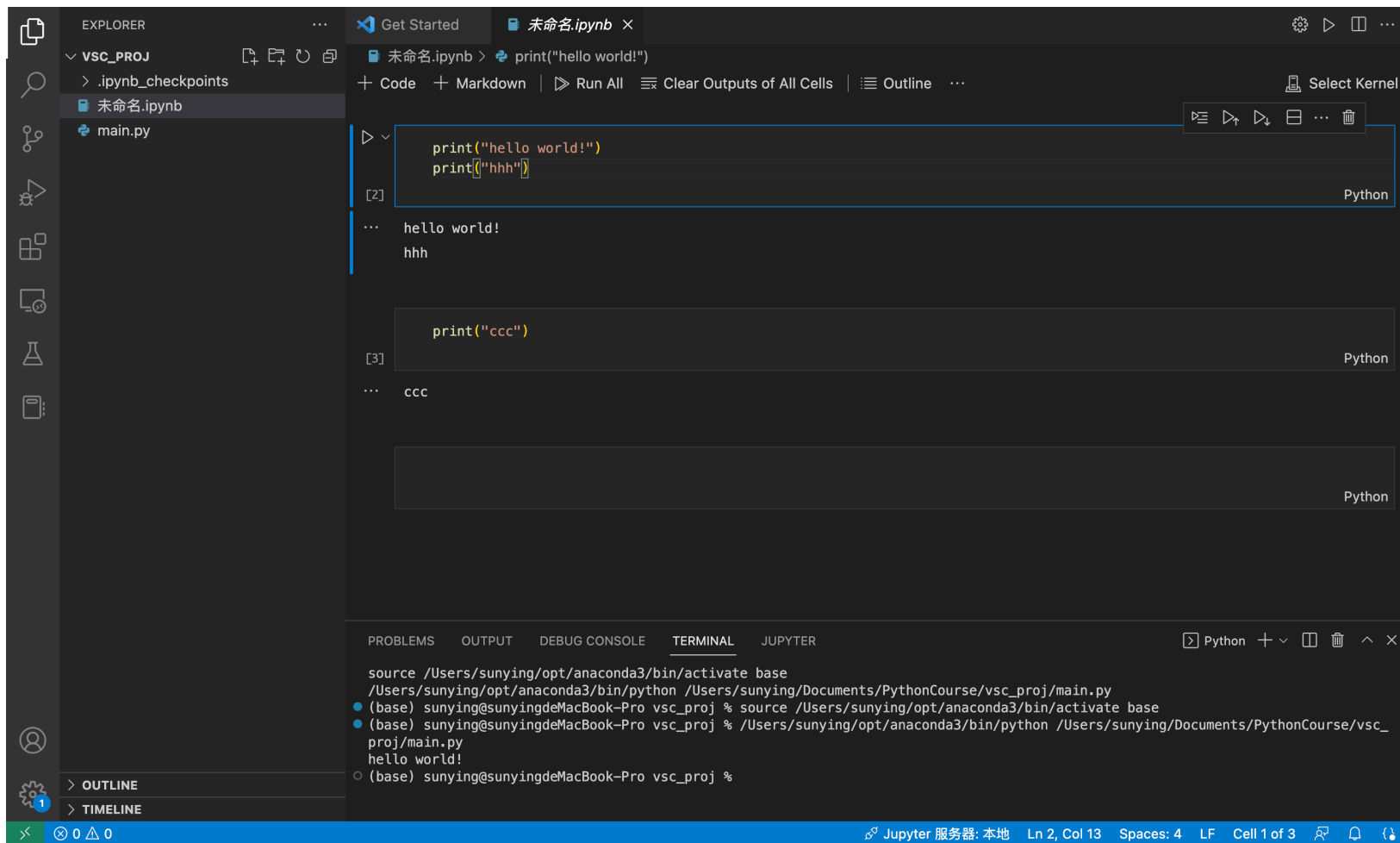
只需创建 ipynb 文件



仅限专业版

Jupyter Notebook (VS Code)

只需创建 ipynb 文件



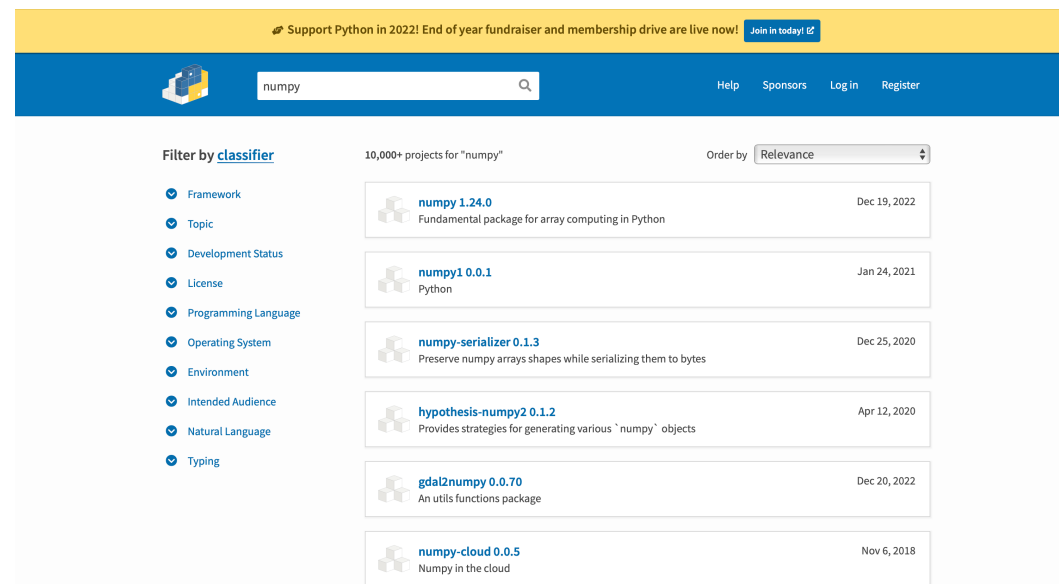
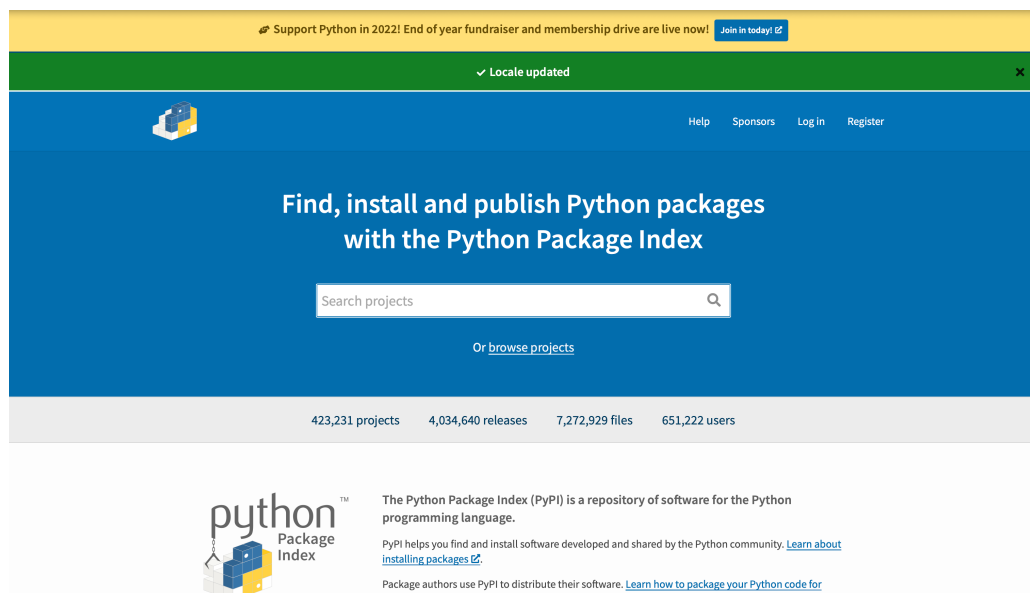
The screenshot displays the VS Code interface with a Jupyter Notebook open. The Explorer sidebar on the left shows a project named 'VSC_PROJ' containing a file 'main.py' and a directory '.ipynb_checkpoints'. The notebook has three cells, each with a 'Python' kernel. The first cell contains the code `print("hello world!")` and `print("hhh")`, with its output showing 'hello world!' and 'hhh'. The second cell contains `print("ccc")`, with its output showing 'ccc'. The third cell is empty. The terminal at the bottom shows the command to activate the base environment and run the notebook:

```
source /Users/sunying/opt/anaconda3/bin/activate base
/Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
(base) sunying@sunyingdeMacBook-Pro vsc_proj % source /Users/sunying/opt/anaconda3/bin/activate base
(base) sunying@sunyingdeMacBook-Pro vsc_proj % /Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
hello world!
```

The status bar at the bottom indicates the Jupyter server is running locally, and the current cell is Cell 1 of 3.

PyPI - Python 包索引

Python 包索引（PyPI）是 Python 的官方第三方软件仓库，作为中心枢纽，供开发者发布并与社区分享包和库。



<https://pypi.org>

PyPI - Python 包索引

安装并使用 PyPI 中的包：pip（随 Python 一并安装）

安装：运行 `pip install <package-name>`

```

pip3 --version
pip 25.2 from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pip (python 3.13)

python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    import numpy
ModuleNotFoundError: No module named 'numpy'
>>> exit()

pip3 install numpy
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Collecting numpy
  Downloading https://mirrors.aliyun.com/pypi/packages/78/45/d4698c182895af189c463fc91d70805d455a227261d950e4e0f1310c2550/numpy-2.3.2-cp313-cp313-macosx_14_0_arm64.whl (5.1 MB)
    5.1/5.1 MB 6.5 MB/s 0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.3.2

python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>>
```



GitHub Copilot 配置

- Copilot 在编码环境中提供 AI 驱动的代码建议与对话辅助。
- 它可辅助代码补全、代码解释、调试、测试生成和文档编写。
- Copilot 能帮你写代码，但理解、测试和验证代码的责任在你。

GitHub Copilot 通用设置流程

1. 创建或登录 GitHub 账户。
2. 申请 GitHub Education / Copilot Student 权限。
3. 安装 IDE 或 notebook 环境。（多数同学已经完成）
4. 安装 GitHub Copilot 扩展/插件。
5. 在 IDE 中用 GitHub 登录。
6. 测试行内建议和 Copilot Chat。

- 前往 GitHub Education 学生权益页面。
<https://github.com/education/students>
- 使用你的 GitHub 账号登录。
- 使用学校邮箱或所需材料验证学生身份。
- 获批后*，开通 GitHub Copilot Student 权限。
- 使用同一 GitHub 账号在 VS Code 或 PyCharm 中登录。

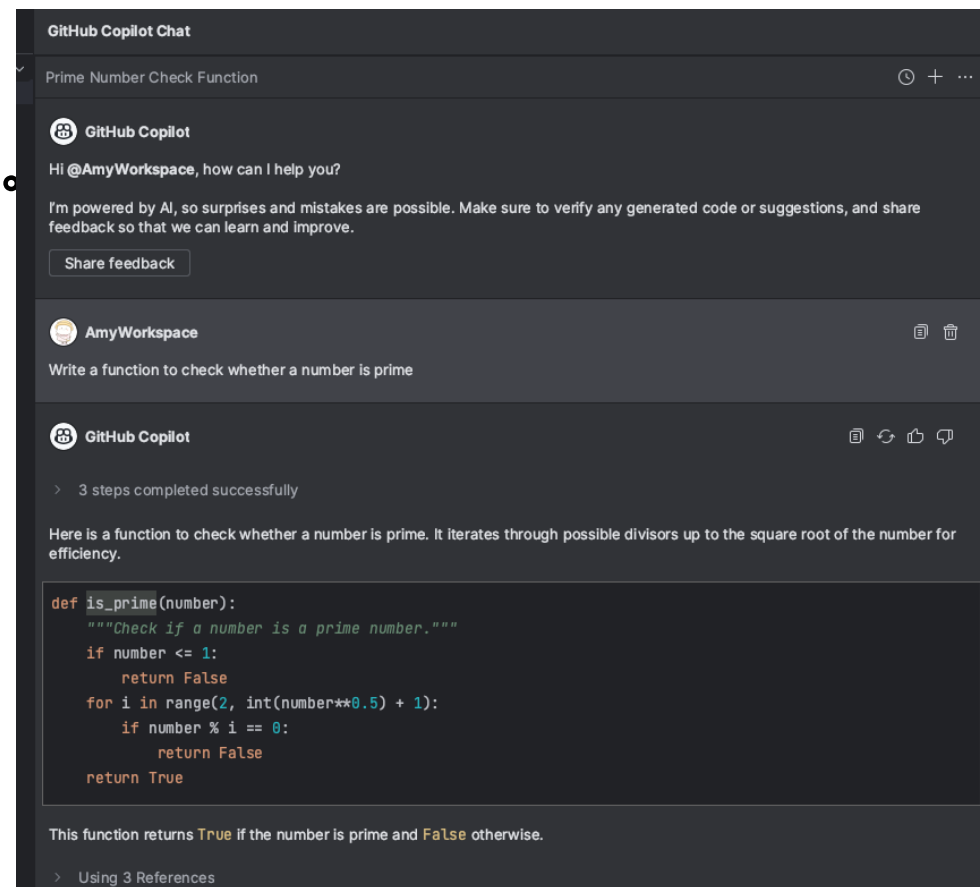
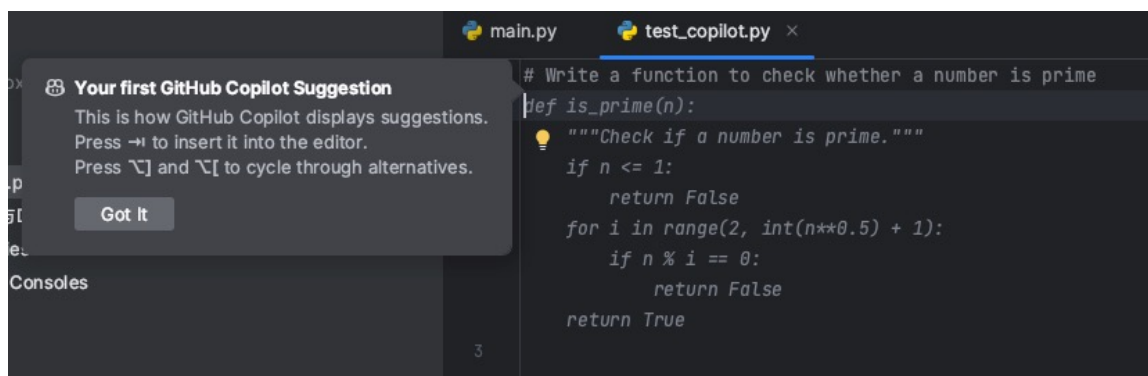
* 审批可能需要时间。

* 也有免费方案可供入门。

中文教程：https://blog.csdn.net/PGJ_168/article/details/150267222

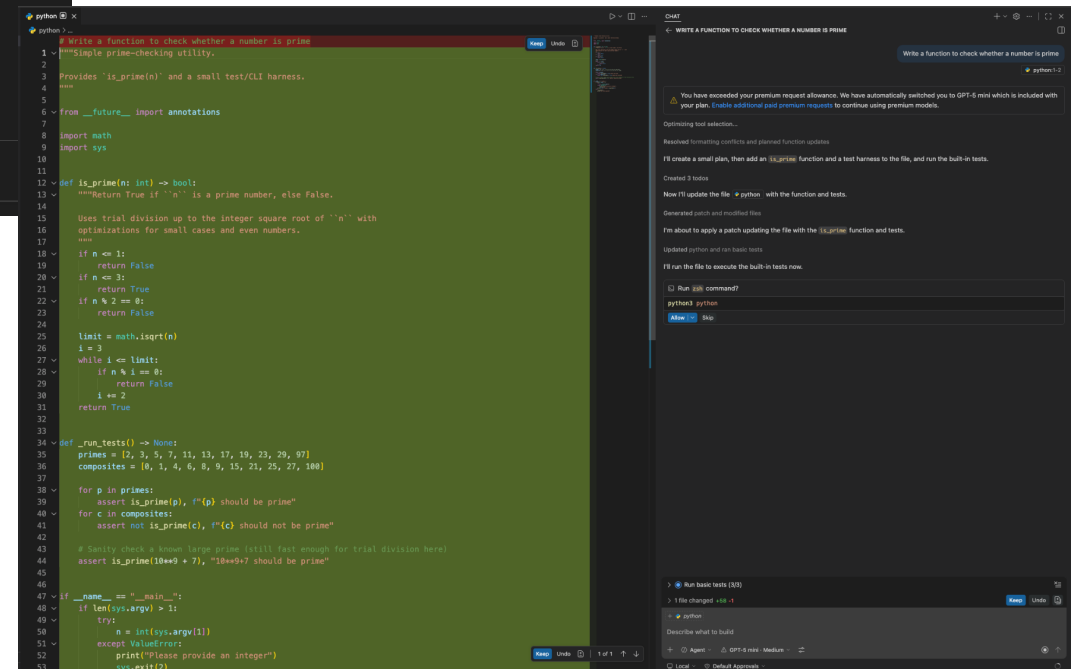
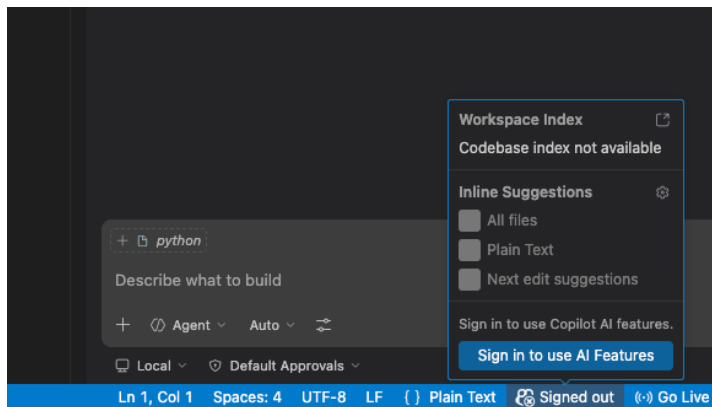
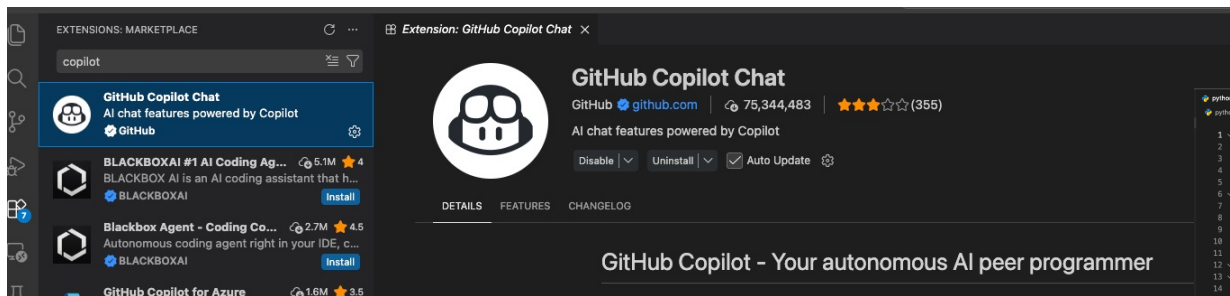
GitHub Copilot + PyCharm

- 先配置 PyCharm
- 打开 Settings / Preferences → Plugins, 安装 Copilot 插件。
- 如有需要请重启 PyCharm。
- 使用你的 GitHub 账号登录。
- 打开一个 Python 文件并测试 copilot 建议。



GitHub Copilot + VS Code

- 先配置 VS Code
- 安装 GitHub Copilot 扩展。
- 使用你的 GitHub 账号登录。
- 打开 .py 文件并测试 Copilot 建议。



- 不支持仅在浏览器中使用的 Jupyter Notebook
- 推荐方法：在 VS Code/PyCharm 中使用 Jupyter Notebook
- 在 IDE 中打开或创建 .ipynb 文件。
- 登录 GitHub。
- 在 notebook 代码单元格中试用 Copilot 建议。

允许

- 用 Copilot 理解示例。
- 用 Copilot 获取编码提示。
- 用 Copilot 调试错误信息。
- 使用 Copilot 生成可能的测试用例。
- 在理解代码之后，再用 Copilot 提升可读性。

不允许

- 提交自己无法解释的代码。
- 未经测试就提交 AI 生成的代码。
- 用 Copilot 绕过独立完成作业。
- 不理解就复制完整解答。

可以使用 AI 工具作为助手，但提交的代码由你全权负责。



简单的 LLM API 调用演示

■ 为何调用 LLM API?

API 调用是 AI 应用与智能体的基本构建块。LLM API 让我们的代码能够使用 AI 能力。

与 Copilot 的关系:

直接调用 LLM API: 由你自己编写请求。

Copilot: 由 IDE/插件为你编写并管理请求。

■ 什么是 LLM API 调用?

你的 Python 程序向 LLM 服务器发送请求。

请求包含：

- API URL、API 密钥、模型名称
- 系统提示、用户提示
- 一些参数，例如 temperature

服务器返回响应，通常为 JSON 格式。

■ 最小示例

```
import requests
import json

url = "https://aigc-api.hkust-gz.edu.cn/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": api_key
}

data = {
    "model": "DeepSeek-V4-Pro",
    "messages": [
        {"role": "system", "content": "You are a helpful coding assistant."},
        {"role": "user", "content": "Write a Python program to print all prime numbers between 0 and 100."}
    ],
    "temperature": 0.7
}

response = requests.post(url, headers=headers, data=json.dumps(data))
```

API 密钥不应硬编码在源代码中。

■ 最小示例：从文件读取 API Key

```
def load_api_key(key_file):  
    with open(key_file, "r", encoding="utf-8") as f:  
        return f.read().strip()  
  
api_key = load_api_key("api_key.txt")
```

- 将密钥保存在 api_key.txt 中。
- 请勿分享密钥。
- 不要把密钥上传到 GitHub。
- 将 api_key.txt 加入 .gitignore。

■ 最小示例：解析并使用响应

```
response_json = response.json()
message = response_json["choices"][0]["message"]["content"]
print(message)
```

调用 LLM API 不仅仅是获取文本。

这是构建 AI 驱动的 Python 应用的第一步。

不要过早把思考外包出去。

1. 先思考

理解问题，设计自己的方案，并写出第一版代码。

2. 再与 AI 对比

让 AI 求解同一问题，并与你的解法对比。

3. 问为什么

不要只向 AI 要答案。还要问：

- 为什么这个解法有效？
- 与我的解法有何不同？
- 哪种实现更易读或更高效？
- 是否存在隐藏的 bug 或边界情况？等等。

4. 把 AI 当作学习伙伴

AI 可以解释、调试、建议替代方案并生成测试——但理解与验证代码的责任在你。

编程“脑力肌肉”需要训练。用 AI 来强化它，而不是替代它。

- 课程信息
- Python 简介
- Python 与 IDE 配置
- GitHub Copilot 配置
- 简单的 LLM API 调用演示

若继续修读本课程，请在下次课前熟悉 Python 环境、选定常用 IDE、配置好 Copilot，并掌握基本的 LLM API 调用。

Python 编程与人工智能

感谢聆听!